

CSC-3TC33

Algorithmique et fondements de l'informatique

Concevoir un programme - partie A

Bertrand Meyer

Automne 2024

Introduction

Notion de problème : quelques exemples

Exemples

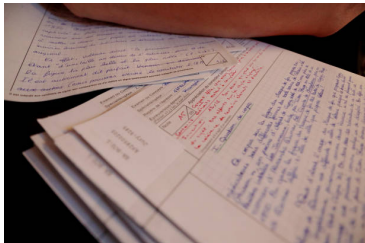
- Préparer une mousse au chocolat pour 2 personnes



Notion de problème : quelques exemples

Exemples

- Préparer une mousse au chocolat pour 2 personnes
- Calculer la somme d'une liste d'entiers
- Trier des copies



Notion de problème : quelques exemples

Exemples

- Préparer une mousse au chocolat pour 2 personnes
- Calculer la somme d'une liste d'entiers
- Trier des copies
- Amener une voiture automatique à destination



Notion de problème : quelques exemples

Exemples

- Préparer une mousse au chocolat pour 2 personnes
- Calculer la somme d'une liste d'entiers
- Trier des copies
- Amener une voiture automatique à destination
- Choisir une liste de lecture musicale



Notion de problème : quelques exemples

Exemples

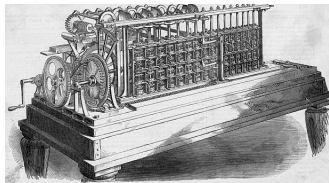
- Préparer une mousse au chocolat pour 2 personnes
- Calculer la somme d'une liste d'entiers
- Trier des copies
- Amener une voiture automatique à destination
- Choisir une liste de lecture musicale

En algorithmique, le terme **problème** désigne la question à résoudre en tout généralité; tandis que le terme **instance** désigne les **données** d'entrée.

Qu'est-ce qu'un algorithme ?

Définition

Un **algorithme** est une suite finie d'étapes non-ambigües qui permet de résoudre n'importe quelle instance d'un problème et qui peut être exécutée de façon automatique.

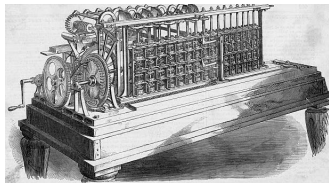


Qu'est-ce qu'un algorithme ?

Définition

Un **algorithme** est une suite finie d'étapes non-ambigües qui permet de résoudre n'importe quelle instance d'un problème et qui peut être exécutée de façon automatique.

Comment décrire un algorithme
ou un programme ?



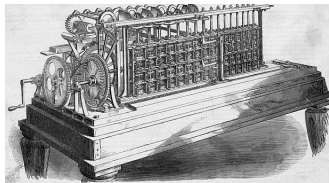
Qu'est-ce qu'un algorithme ?

Définition

Un **algorithme** est une suite finie d'étapes non-ambigües qui permet de résoudre n'importe quelle instance d'un problème et qui peut être exécutée de façon automatique.

Comment décrire un algorithme ou un programme ?

- Donner son principe en langage naturel



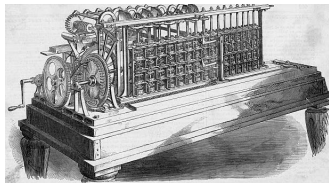
Qu'est-ce qu'un algorithme ?

Définition

Un **algorithme** est une suite finie d'étapes non-ambigües qui permet de résoudre n'importe quelle instance d'un problème et qui peut être exécutée de façon automatique.

Comment décrire un algorithme ou un programme ?

- Donner son principe en langage naturel
- Utiliser du pseudo-code



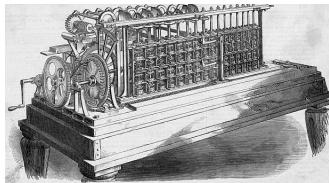
Qu'est-ce qu'un algorithme ?

Définition

Un **algorithme** est une suite finie d'étapes non-ambigües qui permet de résoudre n'importe quelle instance d'un problème et qui peut être exécutée de façon automatique.

Comment décrire un algorithme ou un programme ?

- Donner son principe en langage naturel
- Utiliser du pseudo-code
- Implémenter dans un langage de programmation



Quelques pionnier·es



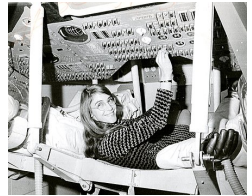
Augusta Ada King, comtesse de Lovelace
(1815+1852)

Première programmeuse
sur la machine de Babbage.



Betty Holberton
(1917+2001)

Une des 6 programmeuse de la
machine ENIAC, aux côtés de Kathleen
Antonelli, Jean Bartik, Marlyn Meltzer,
Frances Spence et Ruth Teitelbaum.



Margaret Heafield Hamilton
(1936)

Créatrice du terme *software engineering*.
Logiciel de guidage d'Apollo.

Nécessité d'un modèle de machine

Machine modèle dotée

- d'une **mémoire**
avec des types élémentaires prédéfinis (entiers, flottants, booléens, etc)
- d'**opérations élémentaires**
lecture et écriture en mémoire, opérations arithmétiques ou booléennes, etc.
- d'un **processeur**
qui effectue les opérations en temps constant.

Conséquence

Permet de parler d'**algorithmes** indépendamment du matériel et du langage de programmation plutôt que de **programmes**.

Caractéristiques d'un algorithme

Caractéristiques d'un algorithme

- **Simplicité**

→ *Compréhension par un humain, réutilisabilité et maintenance du code*

Caractéristiques d'un algorithme

- **Simplicité**

→ Compréhension par un humain, réutilisabilité et maintenance du code

- **Correction** et qualité du résultat

→ méthodes de preuve (système critiques), critères de qualité parfois difficiles à définir.

Caractéristiques d'un algorithme

- **Simplicité**

→ Compréhension par un humain, réutilisabilité et maintenance du code

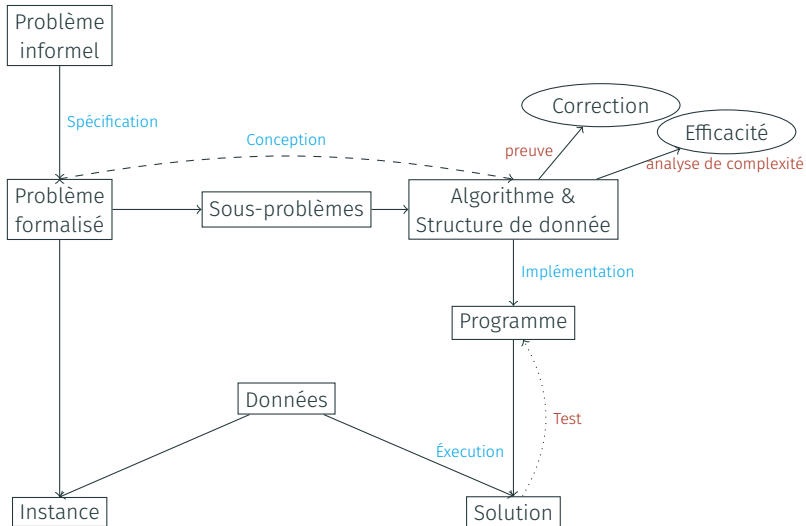
- **Correction** et qualité du résultat

→ méthodes de preuve (système critiques), critères de qualité parfois difficiles à définir.

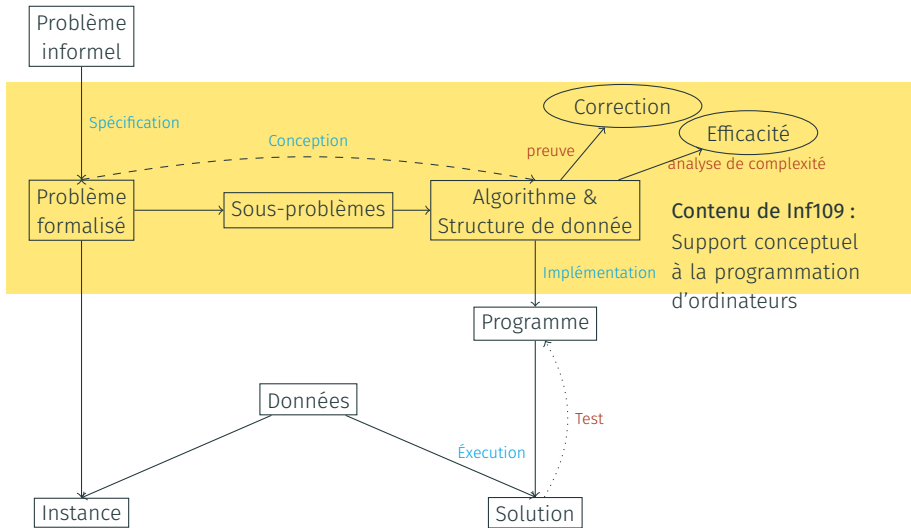
- **Efficacité**, consommation de ressources

- Temps d'exécution
- Place en mémoire
- Consommation d'énergie

D'un problème à la solution : vue d'ensemble



D'un problème à la solution : vue d'ensemble



Spécification, preuve et test

Spécification

La **spécification** d'un problème ou d'un algorithme est la description sans ambiguïté

- des données fournies en entrées,
- des effets sur l'environnement
- et de la valeur de retour



Spécification

La **spécification** d'un problème ou d'un algorithme est la description sans ambiguïté

- des données fournies en entrées,
- des effets sur l'environnement
- et de la valeur de retour

Idéalement : exprimée dans un **langage mathématique**.



Spécification

La **spécification** d'un problème ou d'un algorithme est la description sans ambiguïté

- des données fournies en entrées,
- des effets sur l'environnement
- et de la valeur de retour



Idéalement : exprimée dans un **langage mathématique**.

Indispensable pour **raisonner** sur un programme.

Permet d'utiliser un algorithme en **boîte noire**.

Exemple : le pgcd

Entrées : Entiers a et b

Valeur de retour : Entier d

Effet : Aucun

Préconditions : $a > 0, b > 0, a \geq b$.

Postconditions : $d > 0$;
 d divise a ;
 d divise b ;
si n divise a et b , alors n divise d .

Exemple : le drapeau hollandais

Informellement : dans un tableau à trois couleurs, on souhaite regrouper les cases par couleur.



Exemple : le drapeau hollandais

Informellement : dans un tableau
à trois couleurs, on souhaite
regrouper les cases par couleur.



Passer de



à



Exemple : le drapeau hollandais

Entrées : Tableau T de longueur n dans l'état T_0 .

Valeur de retour : Aucune

Effet : Le tableau T est transformé en T_1

Préconditions : $\forall x \in \llbracket 0, n \rrbracket, T_0[x] \in \{\text{rouge}, \text{blanc}, \text{bleu}\}.$

Postconditions : T_1 est une permutation de T_0

Il existe r et w tels que $0 \leq r \leq w \leq n$ et

$\forall x \in \llbracket 0, r \rrbracket, T_0[x] = \text{rouge},$

$\forall x \in \llbracket r, w \rrbracket, T_0[x] = \text{blanc},$

$\forall x \in \llbracket w, n \rrbracket, T_0[x] = \text{bleu}.$

Définition

Un algorithme **se termine** s'il produit toujours un résultat en temps fini.

Définition

Un algorithme **se termine** s'il produit toujours un résultat en temps fini.

Un algorithme est **partiellement correct** par rapport à une spécification, si pour toute entrée vérifiant les préconditions et si le calcul s'arrête, alors les postconditions sont satisfaites.

Définition

Un algorithme **se termine** s'il produit toujours un résultat en temps fini.

Un algorithme est **partiellement correct** par rapport à une spécification, si pour toute entrée vérifiant les préconditions et si le calcul s'arrête, alors les postconditions sont satisfaites.

Un algorithme est **correct** s'il se termine et est partiellement correct.



Définition

Un **variant** est toute quantité à valeur dans un ensemble bien-fondé^a qui décroît strictement à chaque itération de la boucle.

Un variant permet de démontrer la **terminaison** d'un algorithme

-
- a. Ensemble ordonné dans lequel il n'existe pas de suite strictement décroissante infinie.

Outils de preuve de correction

Définition

Un **variant** est toute quantité à valeur dans un ensemble bien-fondé^a qui décroît strictement à chaque itération de la boucle.

Un variant permet de démontrer la **terminaison** d'un algorithme

a. Ensemble ordonné dans lequel il n'existe pas de suite strictement décroissante infinie.



Définition

Un **invariant** est toute propriété P telle que si P est vérifiée en entrée de boucle, alors P vérifiée en entrée de boucle à l'itération suivante.

Un invariant permet de démontrer la **correction partielle** d'un algorithme



L'algorithme d'Euclide prouvé

```
def gcd(a, b):  
    x, y = a, b  
    while y!=0:  
        temp = y  
        y = x % y  
        x = temp  
    return(x)
```

L'algorithme d'Euclide prouvé

```
def gcd(a, b):  
    x, y = a, b  
    while y != 0:  
        temp = y  
        y = x % y  
        x = temp  
    return(x)
```

$$x = qy + r \text{ avec } 0 \leq r < y$$

L'algorithme d'Euclide prouvé

```
def gcd(a, b):  
    x, y = a, b  
    while y != 0:  
        temp = y  
        y = x % y  
        x = temp  
    return(x)
```

$$x = qy + r \text{ avec } 0 \leq r < y$$

Variant $y \in \mathbb{N}$ décroît strictement

L'algorithme d'Euclide prouvé

```
def gcd(a, b):  
    x, y = a, b  
    while y != 0:  
        temp = y  
        y = x % y  
        x = temp  
    return(x)
```

$$x = qy + r \text{ avec } 0 \leq r < y$$

Variant $y \in \mathbb{N}$ décroît strictement

Invariant On a

$$\text{pgcd}(x, y) = \text{pgcd}(a, b)$$

Le drapeau hollandais codé

1976



Edsger Wybe Dijkstra
(1930+2002)

```
def dutch_flag(T):  
    r, w, b = 0, 0, len(T)-1  
    while w <= b:  
        match T[w]:  
            case "Red":  
                T[r],T[w] = T[w],T[r]  
                r += 1  
                w += 1  
            case "White":  
                w += 1  
            case "Blue":  
                T[w],T[b] = T[b],T[w]  
                b -= 1
```

Le drapeau hollandais exécuté

Situation initiale



Transformation



Le drapeau hollandais exécuté

Situation initiale



Transformation



Le drapeau hollandais exécuté

Situation initiale



Transformation



Le drapeau hollandais exécuté

Situation initiale



Transformation



Le drapeau hollandais exécuté

Situation initiale



Transformation



Le drapeau hollandais exécuté

Situation initiale



Transformation



Le drapeau hollandais exécuté

Situation initiale



Transformation



Le drapeau hollandais exécuté

Situation initiale



Transformation

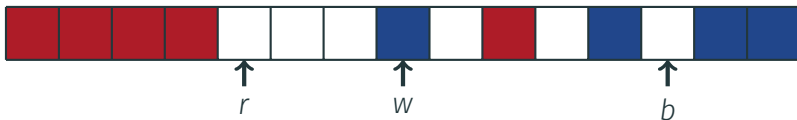


Le drapeau hollandais exécuté

Situation initiale



Transformation



Le drapeau hollandais exécuté

Situation initiale



Transformation

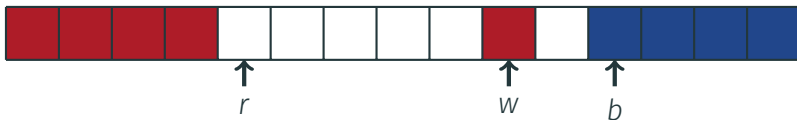


Le drapeau hollandais exécuté

Situation initiale



Transformation

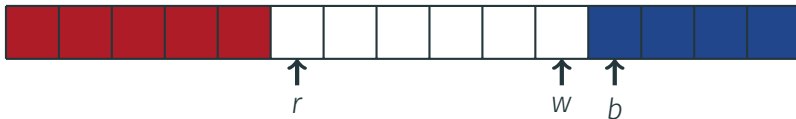


Le drapeau hollandais exécuté

Situation initiale



Transformation

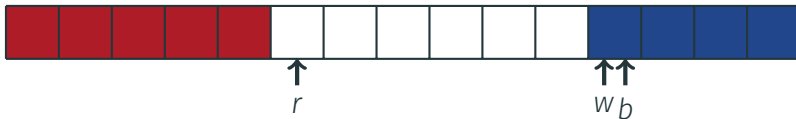


Le drapeau hollandais exécuté

Situation initiale



Transformation

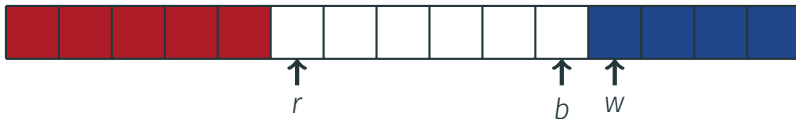


Le drapeau hollandais exécuté

Situation initiale



Transformation



Le drapeau hollandais prouvé

```
def dutch_flag(T):  
    r, w, b = 0, 0, len(T)-1  
    while w <= b:  
        match T[w]:  
            case "Red":  
                T[r],T[w] = T[w],T[r]  
                r += 1  
                w += 1  
            case "White":  
                w += 1  
            case "Blue":  
                T[w],T[b] = T[b],T[w]  
                b -= 1
```

Le drapeau hollandais prouvé

```
def dutch_flag(T):  
    r, w, b = 0, 0, len(T)-1  
    while w <= b:  
        match T[w]:  
            case "Red":  
                T[r], T[w] = T[w], T[r]  
                r += 1  
                w += 1  
            case "White":  
                w += 1  
            case "Blue":  
                T[w], T[b] = T[b], T[w]  
                b -= 1
```

Variant $b - w \in \mathbb{N}$ décroît
strictement

Le drapeau hollandais prouvé

```
def dutch_flag(T):  
    r, w, b = 0, 0, len(T)-1  
    while w <= b:  
        match T[w]:  
            case "Red":  
                T[r], T[w] = T[w], T[r]  
                r += 1  
                w += 1  
            case "White":  
                w += 1  
            case "Blue":  
                T[w], T[b] = T[b], T[w]  
                b -= 1
```

Variant $b - w \in \mathbb{N}$ décroît strictement

Invariant 1 T_{courant} est une permutation de T_{init}

Le drapeau hollandais prouvé

```
def dutch_flag(T):  
    r, w, b = 0, 0, len(T)-1  
    while w <= b:  
        match T[w]:  
            case "Red":  
                T[r], T[w] = T[w], T[r]  
                r += 1  
                w += 1  
            case "White":  
                w += 1  
            case "Blue":  
                T[w], T[b] = T[b], T[w]  
                b -= 1
```

Variant $b - w \in \mathbb{N}$ décroît
strictement

Invariant 1 T_{courant} est une
permutation de T_{init}

Invariant 2 $r \leq w \leq b$

Le drapeau hollandais prouvé

```
def dutch_flag(T):  
    r, w, b = 0, 0, len(T)-1  
    while w <= b:  
        match T[w]:  
            case "Red":  
                T[r], T[w] = T[w], T[r]  
                r += 1  
                w += 1  
            case "White":  
                w += 1  
            case "Blue":  
                T[w], T[b] = T[b], T[w]  
                b -= 1
```

Variant $b - w \in \mathbb{N}$ décroît strictement

Invariant 1 T_{courant} est une permutation de T_{init}

Invariant 2 $r \leq w \leq b$

Invariant 3 $T[0 : r[$ rouge
 $T[r : w[$ blanc
 $T[b + 1 : n[$ bleu

Pour aller plus loin (hors programme inf109)

Dans la vie réelle, on ne fait pas confiance à des preuves humaines, mais on s'appuie sur les **preuves mécanisées**.

Il existe des cadres de programmation dans lesquels le programmeur annote son code avec des préconditions, postconditions, variants ou invariants; la machine se charge de trouver ou contrôler des éléments de preuve qui relient les annotations.

Tester son code

Règle 1 Tester son code

Règle 2 Répéter la règle 1

Tester son code

Règle 1 Tester son code

Règle 2 Répéter la règle 1

Conseils généraux

- Être paranoïaque.
- Un test comprend toujours le jeu d'entrée ET de sortie.
- Les tests se consignent et se documentent.

Tester son code

Règle 1 Tester son code

Règle 2 Répéter la règle 1

Conseils généraux

- Être paranoïaque.
- Un test comprend toujours le jeu d'entrée ET de sortie.
- Les tests se consignent et se documentent.

À quels tests penser

Cas de base : liste vide ou à un élément, entier nul.

Cas limite : indices nuls, égalité dans les inégalités, valeur manquante ou répétée

Branchements : Tester tous les chemins du flot de contrôle.

Ressources et complexité

Définition

Soient $\mathbf{u} = (u_n)_{n \in \mathbb{N}}$ et $\mathbf{v} = (v_n)_{n \in \mathbb{N}}$ deux suites réelles. On définit

- $\mathbf{u} = O(\mathbf{v})$ s'il existe une constante $A > 0$ et un rang $n_0 \in \mathbb{N}$ tel que pour tout indice $n \geq n_0$, $|u_n| \leq A|v_n|$.

Définition

Soient $\mathbf{u} = (u_n)_{n \in \mathbb{N}}$ et $\mathbf{v} = (v_n)_{n \in \mathbb{N}}$ deux suites réelles. On définit

- $\mathbf{u} = O(\mathbf{v})$ s'il existe une constante $A > 0$ et un rang $n_0 \in \mathbb{N}$ tel que pour tout indice $n \geq n_0$, $|u_n| \leq A|v_n|$.
- $\mathbf{u} = \Omega(\mathbf{v})$ s'il existe une constante $A' > 0$ et un rang $n_0 \in \mathbb{N}$ tel que pour tout indice $n \geq n_0$, $|v_n| \leq A'|u_n|$.

Définition

Soient $\mathbf{u} = (u_n)_{n \in \mathbb{N}}$ et $\mathbf{v} = (v_n)_{n \in \mathbb{N}}$ deux suites réelles. On définit

- $\mathbf{u} = O(\mathbf{v})$ s'il existe une constante $A > 0$ et un rang $n_0 \in \mathbb{N}$ tel que pour tout indice $n \geq n_0$, $|u_n| \leq A|v_n|$.
- $\mathbf{u} = \Omega(\mathbf{v})$ s'il existe une constante $A' > 0$ et un rang $n_0 \in \mathbb{N}$ tel que pour tout indice $n \geq n_0$, $|v_n| \leq A'|u_n|$.
- $\mathbf{u} = \Theta(\mathbf{v})$ s'il existe deux constantes $A > 0$, $A' > 0$ et un rang $n_0 \in \mathbb{N}$ tel que pour tout indice $n \geq n_0$, $|u_n| \leq A|v_n|$ et $|v_n| \leq A'|u_n|$.

Rappels mathématiques

Définition

Soient $\mathbf{u} = (u_n)_{n \in \mathbb{N}}$ et $\mathbf{v} = (v_n)_{n \in \mathbb{N}}$ deux suites réelles. On définit

- $\mathbf{u} = O(\mathbf{v})$ s'il existe une constante $A > 0$ et un rang $n_0 \in \mathbb{N}$ tel que pour tout indice $n \geq n_0$, $|u_n| \leq A|v_n|$.
- $\mathbf{u} = \Omega(\mathbf{v})$ s'il existe une constante $A' > 0$ et un rang $n_0 \in \mathbb{N}$ tel que pour tout indice $n \geq n_0$, $|v_n| \leq A'|u_n|$.
- $\mathbf{u} = \Theta(\mathbf{v})$ s'il existe deux constantes $A > 0$, $A' > 0$ et un rang $n_0 \in \mathbb{N}$ tel que pour tout indice $n \geq n_0$, $|u_n| \leq A|v_n|$ et $|v_n| \leq A'|u_n|$.

Lemme

$$\mathbf{u} = O(\mathbf{v}) \Leftrightarrow \mathbf{v} = \Omega(\mathbf{u}) \quad \mathbf{u} = \Theta(\mathbf{v}) \Leftrightarrow (\mathbf{u} = O(\mathbf{v}) \text{ et } \mathbf{v} = \Omega(\mathbf{u})).$$

Préoccupations

- temps d'exécution nécessaire pour faire aboutir un calcul,
- espace occupé en mémoire pour réaliser un calcul,
- énergie consommée par un calcul,
- quantité de données échangées dans le cas où le calcul se fait sur des unités distribuées.

Taille d'une instance

Génériquement, la **taille d'une instance** I est le nombre $|I|$ de bits nécessaires à la représenter ou un ordre de grandeur de $\Theta(|I|)$.

Taille d'une instance

Génériquement, la **taille d'une instance** I est le nombre $|I|$ de bits nécessaires à la représenter ou un ordre de grandeur de $\Theta(|I|)$.

Exemple (Euclide)

Entrées : deux entiers a et b de taille $\lceil \log_2(a+1) \rceil$ et $\lceil \log_2(b+1) \rceil$ bits.

Taille d'une instance

Génériquement, la **taille d'une instance** I est le nombre $|I|$ de bits nécessaires à la représenter ou un ordre de grandeur de $\Theta(|I|)$.

Exemple (Euclide)

Entrées : deux entiers a et b de taille $\lceil \log_2(a + 1) \rceil$ et $\lceil \log_2(b + 1) \rceil$ bits.

Exemple (Drapeau hollandais)

Entrée : tableau T de longueur n à trois valeurs fixes de type `string` : taille d'instance $\Theta(n)$.

Il n'est pas crucial de compter précisément la place prise par `"Red"`, `"White"`, etc.

Taille d'une instance

Génériquement, la **taille d'une instance** I est le nombre $|I|$ de bits nécessaires à la représenter ou un ordre de grandeur de $\Theta(|I|)$.

Exemple (Euclide)

Entrées : deux entiers a et b de taille $\lceil \log_2(a+1) \rceil$ et $\lceil \log_2(b+1) \rceil$ bits.

Exemple (Drapeau hollandais)

Entrée : tableau T de longueur n à trois valeurs fixes de type `string` : taille d'instance $\Theta(n)$.

Il n'est pas crucial de compter précisément la place prise par `"Red"`, `"White"`, etc.

Exemple (Graphe)

Entrée : graphe $G = (V, E)$, avec $n = \text{Card}(V)$ et $m = \text{Card}(E)$.

Selon la représentation, la taille d'une instance est $\Theta(n + m)$ (liste d'adjacence), $\Theta(n^2)$ (matrice) ou $\Theta(m)$ (liste d'arêtes).

On sous-entend qu'on peut adresser un tableau de taille n en $O(1)$

Complexité d'un algorithme

Définition

On appelle **complexité en temps** tout ordre de grandeur du nombre d'opérations élémentaires en fonction de la taille de l'instance à traiter.

Soit $f_{\mathcal{A}}(l)$ le nombre d'opérations élémentaires quand l'algorithme $\mathcal{A}$ s'applique à l'instance l .

- Complexité **dans le pire des cas** :

$$c_{\mathcal{A}}(n) = \max_{|l|=n} f_{\mathcal{A}}(l).$$

- Complexité **dans le meilleur des cas** :

$$c_{\mathcal{A}}(n) = \min_{|l|=n} f_{\mathcal{A}}(l).$$

- Complexité **en moyenne** :

$$c_{\mathcal{A}}(n) = \mathbb{E} [f_{\mathcal{A}}(l) | |l| = n].$$



Complexité de l'algorithme d'Euclide

```
def gcd(a, b):  
    x, y = a, b  
    while y != 0:  
        temp = y  
        y = x % y  
        x = temp  
    return(x)
```

Lemme

Soient $a \geq b$ et $d = \text{pgcd}(a, b)$. Si l'algo passe k fois dans le **while**, alors

$$a \geq dF_{k+1} \text{ et } b \geq dF_k$$

où $(F_k)_{k \in \mathbb{N}}$ est la suite de Fibonacci.

Donc $a, b = \Omega(\phi^k)$ où $\phi = \frac{1+\sqrt{5}}{2}$.

Complexité de gcd en

$O(\log_2 a + \log_2 b)$ i.e. **linéaire** en la taille de l'entrée.

Complexité du drapeau hollandais

```
def dutch_flag(T):  
    r, w, b = 0, 0, len(T)-1  
    while w <= b:  
        match T[w]:  
            case "Red":  
                T[r],T[w] = T[w],T[r]  
                r += 1  
                w += 1  
            case "White":  
                w += 1  
            case "Blue":  
                T[w],T[b] = T[b],T[w]  
                b -= 1
```

Toutes les lignes de calcul sont en $O(1)$.

Le variant passe par n valeurs.

Complexité de `dutch_flag` en $O(n)$ i.e. **linéaire** en la taille de l'entrée.